Gunter & Scott 1990 — Inventory of Definitions and Theorems

Source: **C. A. Gunter and D. S. Scott, “Semantic Domains,”** *Handbook of Theoretical Computer Science* Vol. B, North-Holland, 1990, pp. 633–674 (`../papers/Gunter Scott 1990.pdf`).

The work list for the Lean formalization: every definition and every one of the paper’s **30 numbered results** (Theorems / Lemmas / Proposition 1–30), in paper order, matched to its Lean equivalent.

Progress (as of r0009, 2026-0806)

#	Quantity	Done	Remaining	Of
1	Definitions to define	**7**	6	≈13
2	**Numbered** results complete	**2** (Lem 4, Thm 7)	26	28
3	Mathlib foundations reused	12	—	12
4	`sorry` in the development	—	**0**	—

Row 2 counts only the paper’s 30 **numbered** results (Theorems / Lemmas / Proposition 1–30) and so understates what is proved. The development contains 55 theorems, 15 definitions and 12 instances; of the theorems, these **eight are claims the paper makes in prose** rather than as numbered results, and all eight are formally verified:

#	Paper claim	Where	Lean
1	“the compact elements [of `P N`] are just the finite subsets of `N`”	p. 9	`isCompactElement_iff_finite`
2	“`P N` … is a domain”	p. 9	`instance : Domain (Set X)`
3	“`D → E` is a … cpo”	Thm 7 proof	`instance : CompletePartialOrder (ScottHom α β)`
4	“… bounded complete … whenever `E` is”	Thm 7 proof	`instance : BoundedComplete (ScottHom α β)`
5	“step(s) … is continuous”	Thm 7 proof	`scottContinuous_stepFun`
6	“… and compact in the ordering on `D → E`”	Thm 7 proof	`isCompactElement_step`
7	“they form a basis for `D → E`”	Thm 7 proof	`instance : IsAlgebraic (ScottHom α β)`
8	every compact function is a *finite* join of step functions	Thm 7 proof, implicit	`exists_finite_isLUB_of_isCo`

Six of those eight are the body of **Theorem 7**, which is now **complete** — all four conjuncts of its conclusion (cpo, bounded complete, algebraic, countably based) are formally verified, as `ScottHom.isBoundedCompleteDomain_scottHom`.

It is proved under **weaker hypotheses than the paper states**: bounded completeness of `D` is never used. `D` need only be a domain. The function space is a cpo for any preordered `D`, algebraic when `D` and `E` are, bounded complete because `E` is, and countably based because `D` and `E` are.

The remaining ~47 theorems are supporting API: the ≪ calculus, the `compactsBelow` machinery, the pointwise order and suprema on `D → E`, and the step-function adjunction. The paper assumes or elides all of it.

4 of ≈13 definitions are formally verified and building, in 6 modules, 985 lines, 0 `sorry`, 0 warnings:

#	Round	Module	Contents
1	r0003	`ScottDomains/WayBelow.lean`	way-below ≪ at `[Preorder α]`, 7 theorems; `x ≪ x ↔ IsCompactElement x` holds by `Iff.rfl`

#	Round	Module	Contents
2	r0004	`ScottDomains/Domain.lean`	`IsAlgebraic`, `Domain` (with the paper's countable-basis condition), `BoundedComplete`, 8 theorems, `Domain Prop`
3	r0005	`ScottDomains/Powerset.lean`	the paper's `P N` (p. 9): compacts of `Set X` are exactly the finite subsets, hence `Domain (Set ℕ)` — the nondegenerate witness
4	r0006–r0007	`ScottDomains/ScottHom.lean`	the continuous function space `D → E`: `ScottHom`, the pointwise order, `CompletePartialOrder`, and `BoundedComplete` when `E` is — **Theorem 7's first sentence in full**
5	r0008	`ScottDomains/StepFunction.lean`	the single step function `step k e`: continuity (from `k` compact), the adjunction `step k e ≤ f ↔ e ≤ f k`, and compactness in `D → E` (from `e` compact)
6	r0009	`ScottDomains/FunctionSpaceDomain.lean`	**`D → E` is algebraic** — the paper's "they form a basis for `D → E`"; the two halves of `IsAlgebraic` use disjoint hypotheses (directedness needs only `E` bounded complete; the lub needs only `D`, `E` algebraic)
7	r0010	`ScottDomains/CompactFunction.lean`	every compact function is a **finite** join of step functions — the finiteness half of the basis claim
8	r0011	`ScottDomains/FunctionSpaceCountable.lean`	`K(D → E)` is countable, hence `Domain (ScottHom α β)` — **Theorem 7 complete**
9	r0012	`ScottDomains/NormalSubposet.lean`	the normal-subposet relation ◁ and **Lemma 4**, all four parts
10	r0012–r0013	`ScottDomains/Projection.lean`	embedding–projection pairs, projections, the paper's "an embedding is an injection, a projection is a surjection"; **`im(p)` is a cpo** and **finitary projections**

Next: **Lemma 5** — the compact elements of `im(p)` are `im(p) ∩ K(D)`, and `im(p) ∩ K(D) ◁ K(D)` — then **Theorem 6**, the isomorphism between normal substructures of `K(D)` and the finitary projections `Fp(D)`.

Work counts

- **Reuse from Mathlib — no work (12):** poset, directed, cpo, ⊥, monotone, continuous, Fixed-Point Theorem (1) & operator, Schröder–Bernstein (2), algebraic lattice, product, λ-notation.
- **Generalize / adapt (4):** compact element (`IsCompactElement` — its *definition* is already stated at `[PartialOrder α]` and so applies to a dcpo verbatim; what is `CompleteLattice`-only is every lemma about it, so the dcpo API must be re-proved), sum, lift (`WithBot`/`Sum`), ideal completion (`Order.Ideal`).
- **Definitions to define — new (≈13; 4 done in r0003–r0004, 9 remaining):** way-below ≪ (**done** — `ScottDomains/WayBelow.lean`), algebraic cpo, **domain**, bounded-complete (**all three done** — `ScottDomains/Domain.lean`), embedding–projection pair, (finitary) projection, normal subposet, effective presentation, smash product, the three powerdomains (Hoare / Smyth / Plotkin), bifinite / Plotkin order, and `D∞`.
- **Theorems to prove (28):** 28 of the paper's 30 numbered results — Theorems 3, 6, 7, 11, 12, 14, 16, 18, 21, 22, 25, 26, 27, 29; Lemmas 4, 5, 8–10, 13, 17, 19, 20, 23, 24, 28, 30; Proposition 15. Only Theorems 1 & 2 come free from Mathlib.

Bottom line: ≈13 definitions to define + 28 results to prove, on top of 12 reused Mathlib foundations. After r0004: **9 definitions + 28 results** remain.

Lean column legend: ✓ reuse Mathlib (name given) · ~ partial (Mathlib has a related or lattice-only version — generalize) · ✗ define / prove (absent from Mathlib v4.32.2, confirmed by grep).

> Statements are paraphrased/de-garbled from the PDF (1990 Type-3 fonts render `→` as `!`, drop `fi` ligatures, mangle math), so read them as a guide to *what* each result is. Notation: `⊑` order, `⨆` directed sup, `⊥` bottom, `≪` way-below, `K(D)` compacts, `Fp(D)` finitary projections.

§2 Recursive definitions of functions

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
2.1	Def	—	**poset** (partially ordered set)	✓ `PartialOrder`
2.1	Def	—	**directed** subset `M`: every finite `u ⊆ M` has an upper bound in `M`	✓ `DirectedOn` / `IsDirected`
2.1	Def	—	**cpo**: poset in which every directed `M` has a lub `⨆M`	✓ `CompletePartialOrder` (chains: `OmegaCompletePartialOrder`)
2.1	Def	—	**bottom** `⊥` (least element)	✓ `OrderBot`
2.1	Def	—	**monotone** function	✓ `Monotone` / `OrderHom`
2.1	Def	—	**continuous**: monotone, `f(⨆M) = ⨆f(M)` for directed `M`	✓ `ScottContinuous`
2.1	Thm	1	**Fixed-Point Theorem**: `f : D → D` continuous ⟹ least fixed point `⨆_n fⁿ(⊥)`	✓ `OrderHom.lfp` (Kleene: `OmegaCompletePartialOrder`)
2.2	Thm	2	**Schröder–Bernstein** for sets	✓ `Function.Embedding.schroederBernstein`
2.3	Def	—	**fixed-point operator** (uniform)	✓ `OrderHom.lfp` / `LawfulFix`
2.3	Thm	3	The standard operator is the **unique uniform** fixed-point operator	✗ prove

§3 Effectively presented domains

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
3.1	Def	—	**compact element** `x`: `x ⊑ ⨆M` (dir.) ⟹ `x ⊑ y` some `y∈M`; `K(D)`	~ `IsCompactElement` — def is `[PartialOrder]`, so reusable on a dcpo; its lemmas are all `CompleteLattice`-only
3.1	Def	—	**way-below** `≪` (approximation relation behind compactness)	✓ `ScottDomains.WayBelow` (r0003) — `x ≪ x ↔ IsCompactElement x` by `Iff.rfl`
3.1	Def	—	**algebraic** cpo: `x = ⨆{x'∈K(D) : x'⊑x}` (directed)	✓ `ScottDomains.IsAlgebraic` (r0004)
3.1	Def	—	**domain** = algebraic cpo **whose basis `K(D)` is countable** (the paper's definition, p. 9 — the countability condition was missing from an earlier draft of this row)	✓ `ScottDomains.Domain` (r0004)
3.1	Def	—	**bounded complete**: `⊥` + every bounded subset has a sup — a *separate* predicate; the paper composes them as "bounded complete domain" (Thm 7, Lem 10, Lem 13, Thm 14), which is the literature's *Scott domain*	✓ `ScottDomains.BoundedComplete` (r0004); the compound is `[Domain α] [BoundedComplete α]`
3.1	Def	—	**(countably based) algebraic lattice**	✓ `IsCompactlyGenerated` (+`CompleteLattice`)

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
3.1	Def	—	**embedding–projection pair** `(g, f)`	✓ `ScottHom.IsEmbeddingProjectionPair` (r0012)
3.1	Def	—	**projection**; **finitary projection** `p`: `p∘p=p⊑id`, `im(p)` a domain	✓ `ScottHom.IsProjection` (r0012), `ScottHom.IsFinitaryProjection` (r0013) — `im(p)` carries a `CompletePartialOrder` via `IsProjection.rangeCompletePartialOrder`
3.1	Def	—	**normal subposet** / substructure	✓ `ScottDomains.IsNormalIn`, notation ◃ (r0012)
3.1	Lem	4	`⟨P(C), ◃⟩` of substructures is a cpo with `{⊥}` least	✓ **proved** (r0012), all four parts
3.1	Lem	5	`p` finitary projection ⟹ compacts of `im(p)` are `{p(x):x∈K(D)}`	✗ prove
3.1	Thm	6	Isomorphism: normal substructures ≅ `Fp(D)` (finitary projections)	✗ prove
3.1	Thm	7	`D,E` bounded-complete domains ⟹ `D → E` bounded-complete domain	✓ **proved** (r0006–r0011) — `ScottHom.isBoundedCompleteDomain_scottHom`; `D` bounded complete is not needed
3.2	Def	—	**effective presentation** `d : ℕ → K(D)`; **effectively presented domain**	✗ define

§4 Operators and functions

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
4.1	Def	—	**product** `D × E`	✓ `Prod` (order/cpo instances)
4.2	Def	—	**Church's λ-notation** (continuous abstraction)	✓ `OrderHom` / ωCPO `ContinuousHom`
4.3	Def	—	**smash product** `D ⊗ E`	✗ define
4.4	Def	—	**sum** `D + E`; **lift** `D⊥`	~ `Sum` / `WithBot`,`Part` (partial)
4.x	Lem	8	Currying/iso laws for function spaces over `D,E,F`	✗ prove
4.x	Lem	9	Product/function-space iso laws over `D,E,F`	✗ prove
4.5	Lem	10	`D,E` bounded complete ⟹ `→,×,⊗,+,()⊥` bounded complete	✗ prove
4.5	Thm	11	**Ideal completion** of a countable pre-order is a domain (all domains so arise)	~ `Order.Ideal` exists → prove
4.5	Thm	12	Initiality of a continuous algebra satisfying axioms `T`	✗ prove
4.5	Lem	13	`D` bounded complete ⟹ powerdomains `D],D[` bounded complete	✗ prove
4.5	Thm	14	Equivalent characterizations of an (algebraic/BC) domain	✗ prove

§5 Powerdomains

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
5.1	Def	—	**powerdomain** (non-deterministic outcomes)	✗ define

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
5.2	Def	—	**Hoare (lower)**, **Smyth (upper)**, **Plotkin (convex)** powerdomains	✗ define
5.3	—	—	Universal & closure properties (see Lem 13, 28, 30)	✗ prove

§6 Bifinite (SFP) domains

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
6.1	Def	—	**Plotkin order** / **bifinite (SFP) domain**: bilimit of finite pointed posets	✗ define
6.1	Prop	15	Every bounded-complete domain is bifinite	✗ prove
6.1	Thm	16	`D` bifinite ⟹ `Fp(D)` is an algebraic lattice	✗ prove
6.2	Lem	17	`D,E` bifinite ⟹ `→,×,⊗,+,()⊥` bifinite (incl. function space)	✗ prove
6.2	Thm	18	If `D` and `D → D` are domains, then `D` is bifinite	✗ prove
6.2	Lem	19	closure `r:D→D` (`r∘r=r⊒id`) ⟹ `im(r)` is a domain	✗ prove
6.2	Lem	20	`D` domain ⟹ `Fc(D)` (finitary closures) is a cpo	✗ prove

§7 Recursive definitions of domains (universal domain, `D∞`)

§	Kind	Ref	Concept / statement	In Lean / Mathlib?
7	Def	—	**recursive domain equation**; **universal domain** / `D∞` (inverse limit)	✗ define
7	Thm	21	`F` representable over cpo `U` ⟹ a domain `D` with `D ≅ F(D)`	✗ prove
7	Thm	22	Any countably-based algebraic lattice `L`: a closure `r : P(ℕ) → L`	~ `IsCompactlyGenerated` → prove
7	Lem	23	The function-space operator is representable over `P(ℕ)`	✗ prove
7	Lem	24	`U` cpo; × and → representable over `U` ⟹ (setup for universality)	✗ prove
7	Thm	25	`U` non-trivial domain representing ×,→ ⟹ `U` **universal**	✗ prove
7	Thm	26	Any signature `(s₁,…,s_n)`: combinators `F₁,…,F_n` solving the equations	✗ prove
7	Thm	27	Any bounded-complete `D`: a projection of the universal domain onto `D`	✗ prove
7	Lem	28	Operators `→,×,⊗,+,()⊥,()],()[` representable over `U`	✗ prove
7	Thm	29	`D` bifinite ⟹ `D+` bifinite; solving `D ≅ D+`	✗ prove
7	Lem	30	Operators `→,×,⊗,+,()⊥,()],()[` p-representable over universal bifinite `V`	✗ prove

---

Tally (matched): ✓ reuse Mathlib ≈ **12** (poset, directed, cpo, ⊥, monotone, continuous, fixed-point Thm 1 & operator, Schröder–Bernstein, algebraic lattice, product, λ-notation) · ~ partial ≈ **4** (compact element, sum/lift, ideal completion

Thm 11, Thm 22) · ✗ define / prove ≈ **44**, of which 4 are done (≪ r0003; algebraic, domain, bounded-complete r0004) → **40 remaining —** the bifinite / powerdomain / `D∞` development and all 28 numbered results.

What is done, and what is next, is listed under **Progress** at the top of this file.